

Observability on Google Cloud — Logs, Metrics, Traces, Alerts

The service is running. How do you know it is healthy — and how do you find out why when it is not?

WHAT WE'LL COVER

Five chapters, four labs

- 01** Seeing what the service is doing
- 02** Cloud Logging
- 03** Cloud Monitoring
- 04** Alerting and SLOs as measurement
- 05** Tracing and following one bad request

01

Seeing what the service is doing

Three signals, what each one answers, what Google Cloud collects for you with no setup, and what you pay for.

01 Seeing what the service is doing

02 Cloud Logging

03 Cloud Monitoring

04 Alerting and SLOs as measurement

05 Tracing and following one bad request

SEEING WHAT THE SERVICE IS DOING

Monitoring answers questions you predicted; **observability** lets you ask the ones you did not

Monitoring	Observability
questions decided in advance	questions asked after the fact
metrics on a dashboard, thresholds that page	logs and traces with enough context to explain one request
'is it healthy?'	'why was it unhealthy for that user at 14:02?'
you build it: chapters 3–4	the platform gives most of it: chapters 2 and 5

The test: can you answer a question you did not plan for, from what was already recorded, without deploying anything? If you need a new log line first, the system was not observable.

THE RULE

Monitoring tells you that something is wrong; **observability** lets you find out why without a redeploy.

THE THREE SIGNALS

Logs say what happened, metrics say how much, traces say where the time went

Signal	Answers	Shape	Good for	Bad for
Logs	what happened to this request	one entry per event, with fields	the error text, the caller, the exact time	counting; cost at volume
Metrics	how much, how often, how fast	numbers over time, aggregated	dashboards, alerts, SLOs	any single request
Traces	where the time went	a tree of timed spans across services	the slow hop in a chain	aggregate health

```
# one request, three ways
LOG      2026-09-09T14:02:07Z  orders-api  GET /orders  500  812ms  trace=projects/p/traces/4flc...
METRIC   run.googleapis.com/request_count{response_code_class=5xx}  14:02  +1  → the 5xx line on the dashboard ticks up
TRACE    4flc...  orders-web  GET /  (830ms)  └─ orders-api  GET /orders  (812ms)  └─ db query (790ms)  ← here
```

THE RULE

Metrics to notice, logs to read, traces to locate — and the trace id in every log entry is the thread between them.

WHAT GOOGLE CLOUD COLLECTS WITH NO SETUP

On Cloud Run everything on this slide already exists — the work is reading it, not producing it

Platform	Logs	Metrics	Traces	Setup
Cloud Run	request log + stdout/stderr	request count/latency, instances, CPU/mem, startup	request spans, sampled	none
GKE	container stdout/stderr, system logs	pod/node/container via built-in agents	none by default	on by default in Autopilot
Compute Engine	serial console only	CPU, disk, network (hypervisor)	none	Ops Agent for memory, processes, app logs
Every service	Admin Activity audit log	quota, API counts	—	none

```
# a Cloud Run request log entry, as collected — nothing was configured
httpRequest: {requestMethod: GET, requestUrl: https://orders-api-.../orders, status: 500, latency: 0.812s, userAgent: python-requests/2.32}
resource: {type: cloud_run_revision, labels: {service_name: orders-api, revision_name: orders-api-00013-def, location: asia-south1}}
trace: projects/orders-prod/traces/4f1c9e... spanId: 7a2b... severity: ERROR timestamp: 2026-09-09T14:02:07.118Z
```

THE RULE

Cloud Run is observable out of the box — the request log, the metrics and the trace id are there before you look.

THE COST MODEL

You pay for what you ingest and keep — so the lever is what you route, exclude and retain

Product	Billed on	Free of charge	Your levers
Cloud Logging	bytes ingested into log buckets; retention beyond default	an allowance per project per month; the _Required bucket (audit)	exclusions · sinks to Cloud Storage/BigQuery · retention per bucket
Cloud Monitoring	custom and log-based metric samples ingested	all built-in Google Cloud metrics; dashboards; alerting	few, low-cardinality custom metrics
Cloud Trace	spans ingested beyond the free tier	a monthly span allowance	sampling rate (Cloud Run samples for you)

THE HABIT

- Exclude what nobody will read (health checks, debug noise) · route what must be kept to a bucket with a retention you chose · keep metrics few and their labels bounded

THE RULE

Ingestion is the bill — decide what to drop, what to route and how long to keep it, before the first invoice does.

02

Cloud Logging

One entry per request; query it, count it, route it, exclude it — and put SQL over it when a query is not enough.

01 Seeing what the service is doing

02 Cloud Logging

03 Cloud Monitoring

04 Alerting and SLOs as measurement

05 Tracing and following one bad request

ANATOMY OF A LOG ENTRY

Resource, severity, payload, labels, trace — every query you will write names one of these

A REQUEST LOG ENTRY — THE FIELDS YOU WILL QUERY

```
{
  "logName": "projects/orders-prod/logs/run.googleapis.com%2Frequests",
  "resource": { "type": "cloud_run_revision",
    "labels": { "service_name": "orders-api", "revision_name": "orders-api-00013-def", "location": "asia-south1" } },
  "timestamp": "2026-09-09T14:02:07.118Z",    "severity": "ERROR",
  "httpRequest": { "requestMethod": "GET", "requestUrl": "https://orders-api-.../orders",
    "status": 500, "latency": "0.812s" },
  "labels": { "instanceId": "0087..." },
  "trace": "projects/orders-prod/traces/4f1c9e2b...", "spanId": "7a2b...",
  "jsonPayload": { ... }    // or textPayload: "..."    or protoPayload for audit entries
}
```

Three payload kinds

- textPayload — a plain line from stdout
- jsonPayload — a JSON object from stdout; every key is filterable
- protoPayload — audit logs: who called which API

Three log names on Cloud Run: .../requests (one per request), .../stdout and .../stderr (your app), clouddaudit.../activity (changes).

THE RULE

Resource says who, severity says how bad, payload says what, trace says which request — learn the skeleton once.

STRUCTURED LOGGING VERSUS PLAIN TEXT

Print JSON and every field becomes queryable; print a sentence and you get a substring search

PLAIN TEXT

```
order 4471 failed for customer 88 after 812ms
→ textPayload: "order 4471 failed for customer 88 after 812ms"
query: textPayload:"failed for customer 88"    # substring; no
numbers, no fields
```

STRUCTURED (ONE JSON OBJECT PER LINE)

```
{"severity":"ERROR","event":"order_failed","order":4471,"customer":
88,"ms":812,
  "logging.googleapis.com/trace":"projects/p/traces/4flc..."}
→ jsonPayload.event, .order, .customer, .ms    severity: ERROR
trace: linked
query: jsonPayload.event="order_failed" AND jsonPayload.ms>500
```

	Plain text	Structured JSON
filter on a field	no — substring only	yes — any key, with comparisons
severity	guessed (stderr = ERROR)	set by the severity key
linked to the request	no	yes — logging.googleapis.com/trace
metric from it	count of a substring	count, sum or distribution of any field

THE RULE

One JSON object per line with severity and trace inside — the cheapest observability improvement a developer can make.

LOGS EXPLORER AND THE QUERY LANGUAGE

Name the resource, the severity, a field and a time — and read the result before you widen it

FIVE PATTERNS AND THE CLI FORM

```
# by resource and severity
resource.type="cloud_run_revision" AND resource.labels.service_name="orders-api" AND severity>=ERROR
# by request
resource.labels.service_name="orders-api" AND httpRequest.status>=500 AND httpRequest.requestUrl="/orders"
# by payload (structured / plain)
jsonPayload.event="order_failed" AND jsonPayload.ms>500                textPayload:"simulated fault"
# by trace — every service, one request
trace="projects/orders-prod/traces/4flc9e2b..."
# the CLI form, with columns
$ gcloud logging read 'resource.labels.service_name="orders-api" AND httpRequest.status>=500' --limit 5 --freshness=1h \
  --format='table(timestamp,httpRequest.status,httpRequest.latency,resource.labels.revision_name,trace)'
```

= is equality · is substring · >= works on numbers, durations and severities · AND binds tighter than OR — use parentheses. Console: Logging → Logs Explorer; the histogram above the results is the fastest 'when did it start'.

THE RULE

Resource, severity, field, trace, time — five patterns; start narrow and read the histogram before you read the rows.

LOG-BASED METRICS

Turn a log pattern into a number over time — the bridge from chapter 2 to chapters 3 and 4

WORKED EXAMPLE — A COUNTER METRIC FROM A PLAIN LOG LINE; A DISTRIBUTION FROM A STRUCTURED FIELD

```
$ gcloud logging metrics create api_simulated_faults --description="orders-api simulated fault lines" \
  --log-filter='resource.type="cloud_run_revision" AND resource.labels.service_name="orders-api" AND textPayload:"simulated fault"'
Created [api_simulated_faults].
# it is now a metric: logging.googleapis.com/user/api_simulated_faults - chart it, alert on it, put it on the dashboard
# a distribution metric from a numeric field (structured logs only):
$ gcloud logging metrics create order_ms --log-filter='jsonPayload.event="order_done"' --value-extractor='EXTRACT(jsonPayload.ms)' --
  bucket-name=...
```

Kind	Counts	Use for
Counter	entries matching the filter	'how often does this error happen' — the fault line, a business event
Distribution	a numeric field of matching entries	a latency or size the built-in metrics do not have

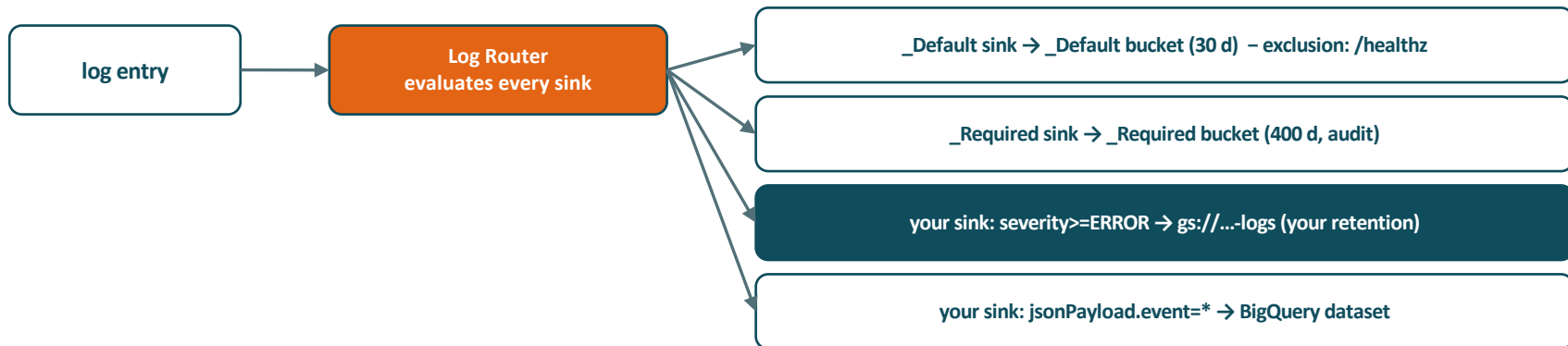
Counts from creation onward only · labels from bounded fields only · every label value is a separate time series you pay for.

THE RULE

A filter plus a counter turns 'this line appeared' into a number you can chart and alert on — from now on, not retroactively.

LOG BUCKETS, RETENTION, SINKS AND EXCLUSIONS

Every entry passes the Log Router once — decide there what is stored, where, for how long, and what is dropped



```
$ gcloud logging sinks create orders-errors storage.googleapis.com/orders-logs-bucket --log-filter='resource.type="cloud_run_revision" AND severity>=ERROR'
$ gcloud storage buckets add-iam-policy-binding gs://orders-logs-bucket --member=$(gcloud logging sinks describe orders-errors --format='value(writerIdentity)') --role=roles/storage.objectCreator
$ gcloud logging sinks update _Default --add-exclusion=name=healthz,filter='httpRequest.requestUrl:"/healthz"'
```

THE RULE

Sinks decide where and how long; exclusions decide what never lands — set both on purpose, before the volume does.

AUDIT LOGS

Admin Activity is always on and free; Data Access is off by default — you met this last session

WORKED EXAMPLE — 'WHAT CHANGED IN THE LAST 30 MINUTES?'

```
$ gcloud logging read 'logName:"cloudaudit.googleapis.com%2Factivity" AND protoPayload.serviceName=("run.googleapis.com" OR
"clouddeploy.googleapis.com")' ' \
--freshness=30m --
format='table(timestamp,protoPayload.authenticationInfo.principalEmail,protoPayload.methodName,protoPayload.resourceName.basename())'
TIMESTAMP      PRINCIPAL EMAIL      METHOD_NAME      RESOURCE
2026-09-09T13:58:41Z bookshelf-deployer@...gserviceaccount.com google.cloud.run.v1.Services.ReplaceService orders-api
2026-09-09T13:58:12Z priya@example.com google.cloud.deploy.v1.CloudDeploy.ApproveRollout r-e7d09a4-to-prod-0001
```

Log	Records	On by default	Where it helps today
Admin Activity	config and policy changes	yes, free, 400 d	'what changed before the spike?'
Data Access	reads/writes of data	no — per service, billed	'who read the secret?' (Session C)
System Event	Google-initiated changes	yes	'did Google migrate/scale something?'

THE RULE

When a graph changes shape, the Admin Activity log for the last thirty minutes is the first thing to read.

LOG ANALYTICS

When a filter is not enough, SQL over the same log bucket — group, join, percentile

LOG ANALYTICS — SQL OVER THE _DEFAULT BUCKET (CONSOLE: LOGGING → LOG ANALYTICS)

```
SELECT resource.labels.service_name AS service, http_request.status AS status, COUNT(*)
AS n
FROM `orders-prod.global._Default._Default`
WHERE timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR) AND resource.type
= 'cloud_run_revision'
GROUP BY 1, 2 ORDER BY 1, 2
```

service	status	n
orders-api	200	4 812
orders-api	500	131
orders-web	200	2 440

Use it for

- counts and ratios grouped by anything
- percentiles from the request log
- joining two services' entries on trace id
- one-off investigations that would otherwise be an export

```
$ gcloud logging buckets update _Default --location=global --enable-analytics # once per bucket
```

THE RULE

A filter finds rows; SQL answers questions about them — enable analytics on the bucket before the day you need it.

LAB 1

Find, filter, follow a trace across two services, count a line, route errors, drop the noise

LAB 1 · LAB MANUAL

Open the lab manual, Lab 1. Every command, console path and expected output is there — nothing is typed from this slide.

- Generate traffic with 30 % faults; read the api's request log; filter to 5xx and to the app's own fault line
- Take one trace id from the web's log and find both services' entries for that page load
- Create a log-based metric on the fault line; sink ERROR entries to a bucket you own; exclude /healthz
- Deploy bookshelf-web 1.1 (forwards the trace header) and enable Log Analytics

About 25 minutes
Kit: gcp-observability-labs.zip
Cloud Shell only — no code

THE RULE

One request, two services, one trace id; a counter, a sink and an exclusion — the logs now work for you.

03

Cloud Monitoring

The metrics that matter on Cloud Run, one dashboard that fits on a page, and an uptime check from outside.

01 Seeing what the service is doing

02 Cloud Logging

03 Cloud Monitoring

04 Alerting and SLOs as measurement

05 Tracing and following one bad request

METRIC TYPES AND THE RESOURCE MODEL

A time series is a metric type, on a monitored resource, with labels — three parts to every chart

A FILTER, READ AS ITS THREE PARTS

```
metric.type="run.googleapis.com/request_count"      # WHAT is measured
resource.type="cloud_run_revision"                 # ON WHAT
resource.label.service_name="orders-api"            # which one
metric.label.response_code_class="5xx"              # split by a metric
label
# → one time series per (service, revision, location, response_code_class, route)
```

Kind	Meaning	Example
Gauge	a value at an instant	container/instance_count
Delta	change over the sample interval	request_count
Cumulative	running total since start	(rare on Cloud Run)
Distribution	a histogram per interval	request_latencies → percentiles

Aligning turns raw points into one value per period (rate, sum, percentile); reducing combines series (sum across revisions). Every chart is filter → align → reduce.

THE RULE

Metric type on a resource with labels — one label combination is one series, and a chart is filter, align, reduce.

THE CLOUD RUN METRICS THAT MATTER

Six built-in metrics answer 'is it healthy' — and they map onto the four golden signals

Metric (run.googleapis.com/...)	Kind	Labels that matter	Golden signal	Watch for
request_count	delta	response_code_class, route	traffic · errors	5xx share rising; traffic dropping to zero
request_latencies	distribution	route	latency	p99 climbing while p50 is flat — queueing
container/instance_count	gauge	state: active / idle	saturation	hitting max-instances; constant cold starts
container/cpu/utilizations · memory/utilizations	distribution	—	saturation	memory near 100 % → OOM kills
container/startup_latencies	distribution	—	latency	slow starts — big image, slow imports
container/billable_instance_time	delta	—	cost	min-instances you forgot

THE RULE

Traffic, errors, latency, saturation — six Cloud Run metrics cover all four, and they are already being collected.

METRICS EXPLORER, ALIGNERS AND AGGREGATORS

Align turns points into one value per minute; reduce combines series — that is all the maths there is

TWO CHARTS AS FILTER → ALIGN → REDUCE (THE DASHBOARD JSON USES THESE EXACT NAMES)

```
# requests per second, one line per service
filter: metric.type="run.googleapis.com/request_count"
resource.type="cloud_run_revision"
align:  ALIGN_RATE over 60s           # counter → per-second rate, per series
reduce: REDUCE_SUM group by resource.label.service_name # all revisions of a
service → one line
# p99 latency of one service
filter: metric.type="run.googleapis.com/request_latencies"
resource.label.service_name="orders-api"
align:  ALIGN_DELTA over 60s         # distribution per minute
reduce: REDUCE_PERCENTILE_99        # one line: the p99
```

Two mistakes

- Averaging a percentile across series — p99 of the mean is not the p99. Reduce distributions with a percentile reducer.
- A rate on a gauge — instance_count has no rate; use max or mean.

Console name	API name	Meaning
Rolling window	alignmentPeriod	the bucket size (60 s)
Rolling window function	perSeriesAligner	how points in a bucket become one value
Time series aggregation · group by	crossSeriesReducer · groupByFields	how series become lines

THE RULE

Every chart is filter, align, reduce — rate for counters, percentile for distributions, and never an average of a percentile.

DASHBOARDS

One page, four golden signals, two services — if it needs scrolling it is not a dashboard, it is a report

traffic

Request rate by service

errors

5xx rate by service

latency

Latency p50 · p95 · p99 (api)

saturation

Active instances by service

Simulated faults (log-based metric)

```
$ gcloud monitoring dashboards create --config-from-file=dashboard.json # the page above, as JSON — edit it in the console afterwards
```

THE RULE

Traffic, errors, latency, saturation and one business number on a single page — a dashboard is for ten seconds under stress.

UPTIME CHECKS

A probe from outside Google, on a schedule, that fails the way a user would — before any user does

WORKED EXAMPLE — A CHECK EVERY MINUTE FROM THREE CONTINENTS, MATCHING THE BODY

```
$ gcloud monitoring uptime create "orders-web page" --resource-type=uptime-url --resource-labels=host=orders-web-123.asia-south1.run.app,project_id=$PROJECT \
  --protocol=https --path=/healthz --port=443 --period=1 --timeout=10 --matcher-content=ok --matcher-type=contains-string
Created uptime check [orders-web page].
# the metric it produces, per probe region:
metric.type="monitoring.googleapis.com/uptime_check/check_passed" checker_location=asia-pacific,europe,usa value: true/false
```

Catches	Because
DNS, certificate, load balancer, ingress mistakes	the probe comes from the internet, like a user
scaled to zero and failing to start	no request would otherwise arrive to be logged
a 200 that is actually an error page	the content matcher

THE RULE

Measure from outside too — an uptime check is the only signal that exists when nothing else does.

THE FOUR GOLDEN SIGNALS

Latency, traffic, errors, saturation — the page, the alerts and the SLO all come from these four

Signal	Question	On Cloud Run	Alert or chart?
Latency	how long — for successes and for failures separately	request_latencies p50/p95/p99	alert (symptom)
Traffic	how much demand	request_count rate	chart (context); alert on zero
Errors	how many fail — explicit 5xx, implicit wrong/slow	request_count 5xx share; uptime check; log-based metric	alert (symptom)
Saturation	how full	instance_count vs max; memory/CPU utilisation	chart (cause); alert near a hard limit

THE RULE

Alert on the two the user feels — latency and errors; chart the two that explain them — traffic and saturation.

LAB 2

One page for two services, an uptime check from outside, and a burst to watch

LAB 2 · LAB MANUAL

Open the lab manual, Lab 2. Every command, console path and expected output is there — nothing is typed from this slide.

- Read request_count in Metrics Explorer: filter, align, reduce, group by response class
- Create the kit's dashboard — rate, 5xx, p50/p95/p99, instances, the Lab 1 metric — and open it
- Create an uptime check on the web page's /healthz from three regions
- Send two bursts and watch instances scale and p99 move; find the deployment marker

About 25 minutes
Kit: gcp-observability-labs.zip
Cloud Shell only — no code

THE RULE

Four signals on one page and a probe from outside — you now see the service instead of loading it.

04

Alerting and SLOs as measurement

Alert on symptoms with a duration; define an SLI, an SLO and an error budget — and read the dent when something breaks.

01 Seeing what the service is doing

02 Cloud Logging

03 Cloud Monitoring

04 Alerting and SLOs as measurement

05 Tracing and following one bad request

ALERTING POLICIES

Condition, threshold, duration, channel — and the duration is what keeps you sane

WORKED EXAMPLE — THE KIT'S POLICY (CONSOLE: MONITORING → ALERTING → CREATE POLICY BUILDS THE SAME THING)

```
{ "displayName": "orders-api - 5xx responses for 2 minutes",
  "documentation": { "content": "First check: a release or IAM change in the last 30 min? (Cloud Deploy rollouts, Admin Activity log)" },
  "conditions": [ { "displayName": "5xx rate > 0.05 req/s for 2 min",
    "conditionThreshold": {
      "filter": "metric.type=\"run.googleapis.com/request_count\" AND resource.label.service_name=\"orders-api\" AND
metric.label.response_code_class=\"5xx\"",
      "comparison": "COMPARISON_GT", "thresholdValue": 0.05, "duration": "120s",
      "aggregations": [ { "alignmentPeriod": "60s", "perSeriesAligner": "ALIGN_RATE", "crossSeriesReducer": "REDUCE_SUM" } ] } },
    "notificationChannels": [ "projects/orders-prod/notificationChannels/1234" ], "alertStrategy": { "autoClose": "1800s" } } ]
$ gcloud monitoring policies create --policy-from-file=alert-5xx.json
```

Condition type	Fires when	Use for
Threshold	value above / below X for the duration	error rate, latency, saturation
Absence	no data for N minutes	a service that stopped reporting
Forecast	predicted to cross X within N hours	disk, quota, certificate expiry

THE RULE

A threshold with a duration, a channel, and one sentence of what to check first — that is an alert someone can act on.

ALERT ON SYMPTOMS, NOT CAUSES

Page a human for what the user feels; chart everything else — alert fatigue is a design failure, not a discipline failure

Page a human (symptom)	Put on the dashboard (cause / context)
5xx share above X for 2 min	CPU and memory utilisation
p99 latency above X for 5 min	instance count, cold starts
uptime check failing from 2+ regions	traffic rate (unless it drops to zero)
SLO burn rate (next slides)	the log-based metric for one error class

ALERT FATIGUE

- twenty policies, most on causes, no duration → pages nobody reads → the real one is missed

TWO TESTS FOR EVERY PAGE

- Is a user affected right now? · Is there something the responder should do? — no to either: chart, do not page

THE RULE

Page for what the user feels and the responder can act on; everything else is a chart — fewer, better alerts.

SLI, SLO AND ERROR BUDGET IN PLAIN WORDS

An SLI is a measurement, an SLO is a promise about it, the error budget is how much promise you have left

SLI — the measurement

- good events ÷ all events
- availability: requests not 5xx
- latency: requests under 500 ms
- measured from the request log or metrics

SLO — the promise

- a target for the SLI over a window
- 99 % of requests succeed, rolling 28 days
- chosen with the product, not by engineering alone
- one or two per service

Error budget — what is left

- 100 % – SLO = allowed failure
- 1 % of 28 days of requests
- spent by incidents and bad releases
- read it; the decision about it is Session B

`SLO 99 % · 28 days · 100 req/min → 4 032 000 requests → budget = 1 % = 40 320 failed requests`

`a 3-minute full outage at 100 req/min = 300 failures = 0.7 % of the budget · 99.99 % would make the budget 403 requests — one bad minute`

THE RULE

Measure a ratio, promise a target over a window, and read what is left — the budget is a number, not a feeling.

SLOS IN CLOUD MONITORING

Define the SLI on a Cloud Run service in four clicks — and the error budget chart appears on its own

1

Cloud Run → service → SLOs → Create SLO

the service is already a Monitoring 'service'

2

SLI: Availability, request-based

good = requests with a non-5xx response; from `run.googleapis.com/request_count`

3

Goal 99 %, rolling window 28 days

or Latency: 95 % under 500 ms — a second SLO

4

Save — the error-budget chart appears

remaining %, consumed by day, current SLI

What the SLO page shows

- current SLI over the window: 99.62 %
- error budget remaining: 62 % — and a chart of how it was spent
- the 28-day burn-down, with each incident as a step
- alerting policies attached to this SLO

Console only: there is no `gcloud` command for SLOs (the API is `monitoring.googleapis.com/v3 services.serviceLevelObjectives`).

THE RULE

The SLO is four clicks on the service — the platform computes the SLI and draws the budget; you only choose the promise.

BURN-RATE ALERTS

Alert on how fast the budget is being spent, not on the raw error rate — fast burn pages, slow burn tickets

Alert	Lookback	Burn rate	Meaning	Response
Fast burn	1 hour	> 10	an outage in progress — at this pace the 28-day budget lasts under 3 days	page
Slow burn	6 hours	> 2	a persistent problem — budget gone in two weeks	ticket, next business day

`burn rate = (error rate over the lookback) ÷ (1 - SLO)`

SLO 99 % → allowed error rate 1 % error rate 10 % over the last hour → burn rate 10 → page
error rate 2 % over the last 6 h → burn rate 2 → ticket

Console: the SLO page → Create alerting policy → burn-rate, lookback, threshold, channel. Two policies per SLO — fast and slow — is the standard shape.

THE RULE

Page on a fast burn, ticket on a slow one — the burn rate measures the promise, not the noise.

WHERE THE DECISIONS LIVE

This session measures; the incident session decides — one line marks the boundary

Today — observability produces	Incident session — response decides
an alert that fires on 5xx for two minutes	whether it is an incident, and its severity
an SLO and the budget remaining	what happens when the budget is gone
a trace pointing at one revision	rollback — traffic, config, secrets, data
the Admin Activity log of what changed	the change record and the postmortem

THE RULE

Today ends when the signal reaches a person with a fact in hand — what they do next is the next session.

LAB 3

A channel, a threshold alert, an SLO with a burn-rate alert — then break the api and receive the email

LAB 3 · LAB MANUAL

Open the lab manual, Lab 3. Every command, console path and expected output is there — nothing is typed from this slide.

- Create an email channel; create the kit's 5xx policy from JSON and read it in the console
- Create a 99 % availability SLO on bookshelf-api in the console; attach a fast-burn alert
- Set the api's fault rate to 100 % for three minutes; watch the incident open; read the email
- Restore; watch the incident close; read the dent in the error budget

About 25 minutes
Kit: gcp-observability-labs.zip
Cloud Shell only — no code

THE RULE

A promise, a budget, and an email that arrived because a number crossed a line — the service now tells you.

05

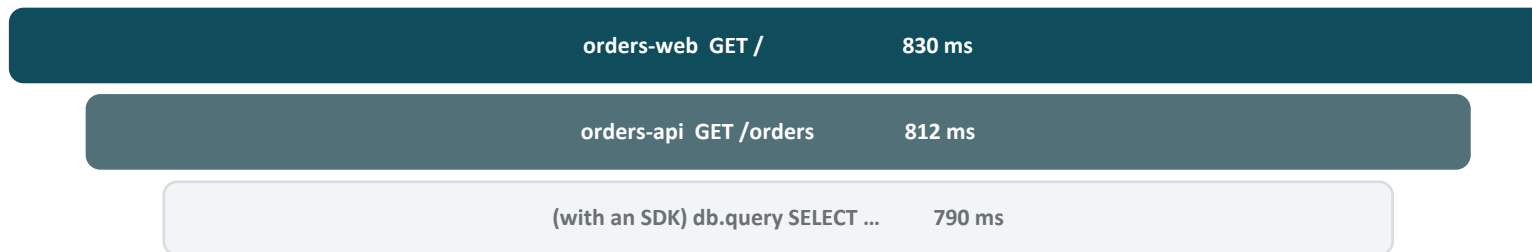
Tracing and following one bad request

One page load as one trace across two services; from a slow span to the log line to the revision that caused it.

- 01 Seeing what the service is doing
- 02 Cloud Logging
- 03 Cloud Monitoring
- 04 Alerting and SLOs as measurement
- 05 Tracing and following one bad request**

DISTRIBUTED TRACING

A trace is one request as a tree of timed spans — and it only crosses a service boundary if the id is passed along



trace 4f1c9e2b... — one request, three spans, parent → child

```
# what Cloud Run puts on the inbound request, and what the web must forward to the api:
traceparent: 00-4f1c9e2b7d3a4c5e8f9a0b1c2d3e4f50-7a2b3c4d5e6f7a8b-01      X-Cloud-Trace-Context: 4f1c9e2b.../8829...;o=1
# bookshelf-web 1.1: copies these two headers onto its outgoing request — that is the entire change
```

THE RULE

A trace crosses a service boundary only if the id travels in the header — forward it, and one request becomes one tree.

CLOUD TRACE ON CLOUD RUN

Request spans are collected for you, sampled — open Trace Explorer, filter by service, sort by latency

WORKED EXAMPLE — A WATERFALL FOR ONE PAGE LOAD, AND THE JUMP TO ITS LOGS

Trace Explorer > filter: service = orders-web · sort: latency desc > open 4f1c9e2b...

```
orders-web  GET / [redacted] 830 ms revision orders-web-00004-abc
orders-api  GET /orders [redacted] 812 ms revision orders-api-00013-def -- here

[Logs for this trace] → orders-api 14:02:07.118 status 500 latency 0.812s revision orders-api-00013-def
```

You get without an SDK	You get with OpenTelemetry (not today)
one span per sampled request per service, with revision, route, status, latency	spans inside the process: database calls, outbound HTTP, functions you name
parent/child across services when the header is forwarded	attributes, events, exemplars linking metrics to traces

THE RULE

Sort by latency, open the slowest, read the gap between the spans, jump to the logs — the revision label is the answer.

ERROR REPORTING AND CLOUD PROFILER

Two tools in one slide each — grouped stack traces, and where the CPU goes inside the process

Tool	Shows	Needs	Use when
Error Reporting	exceptions grouped by stack trace — count, first/last seen, service, revision; new-error notifications	nothing on Cloud Run for Python tracebacks on stderr	'what distinct errors are we throwing, and is one of them new?'
Cloud Profiler	flame graphs of CPU and memory by function, continuously, in production	the profiler agent in the application (a code change)	'this endpoint is slow and nothing outside the process is'

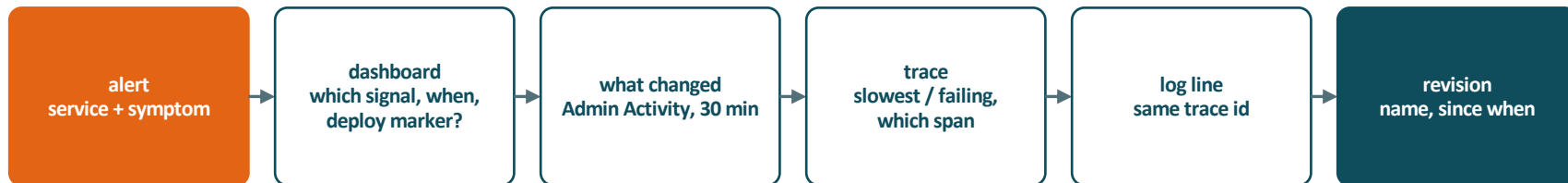
```
Error Reporting > orders-api
ZeroDivisionError: division by zero in compute_eta (app.py:88) count 131 first seen 13:58 last seen 14:06 revisions:
orders-api-00013-def
```

THE RULE

Error Reporting for 'which errors and are they new'; Profiler for 'where inside the process' — know they exist, reach for them when the question fits.

THE DIAGNOSTIC PATH

Alert → dashboard → what changed → trace → log line → revision — and stop there



Step	Built in	Command or click
alert	Lab 3	the email; Monitoring → Incidents
dashboard	Lab 2	Bookshelf — one page; deployment markers on the chart
what changed	Lab 1 / Session C	gcloud logging read 'logName:activity ...' --freshness=30m
trace → log → revision	Lab 1 / Lab 4	Trace Explorer → Logs for this trace → revision label

THE RULE

Six steps from a page to a revision name, each answered by something you built today — then hand it to the responder.

LAB 4

Make the api slow, follow one request through the trace to the log, and name the revision

LAB 4 · LAB MANUAL

Open the lab manual, Lab 4. Every command, console path and expected output is there — nothing is typed from this slide.

- Give the api one instance with concurrency 1 — a config change — and send a burst: requests queue
- In Trace Explorer, sort the web's traces by latency; open the slowest; read the waterfall
- Jump to the logs for that trace id; read both services' entries
- Name the revision, its creation time and what changed — and stop

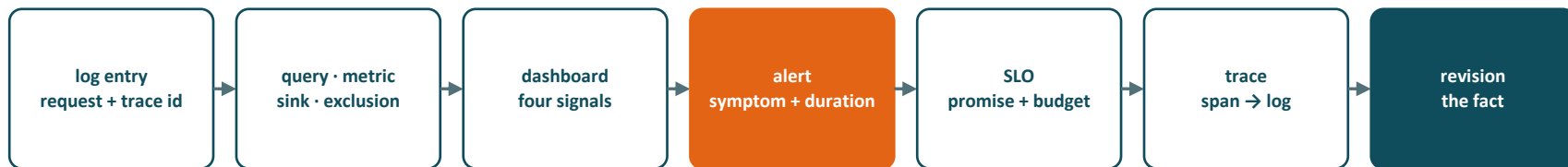
About 20 minutes
Kit: gcp-observability-labs.zip
Cloud Shell only — no code

THE RULE

You went from a slow page to a revision name without guessing — the next session starts from exactly here.

THE WHOLE PATH IN ONE PICTURE

signal → query → dashboard → alert → SLO → trace → revision



You leave with	
a log query for any question and a metric for the one that recurs	plus a sink and an exclusion you chose
one dashboard that fits on a page	traffic, errors, latency, saturation
an alert that emailed you and an SLO with a budget	measured, not felt
a trace that named a revision	the fact the incident session starts from

You can see it now. Next session, it breaks on purpose — and you already know which revision.

Incident response starts from the alert you built, on the dashboard you built, with the trace path you walked — and adds the decisions: severity, roles, rollback, communication, the postmortem.

Leave the alert, dashboard and SLO in place. Kit: [gcp-observability-labs.zip](#) · Lab manual: [gcp-observability-lab-manual.pdf](#)